

COMP41720 Distributed Systems: Architectural Principles LAB 3: Designing for Resilience and Observability

Tools & Environment:

Client Application Language - Java 17

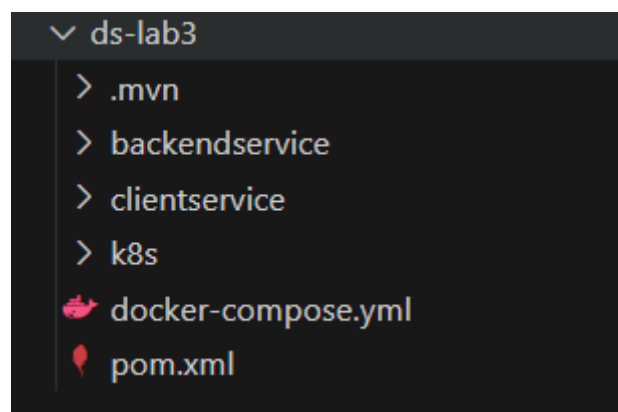
Deployment Environment - Kubernetes with Docker Desktop

Resilience Libraries/ Frameworks - Resilience4j

Chaos Engineering Tool - Chaos Monkey

Part A: Setup & Baseline Distributed Application:

The application is set inside folder *ds-lab3*, there are two services defined, *backendservice* and *clientservice*, both built using a shared *pom.xml* file and composed in docker as shown below in Fig.



The configuration details are briefed in the project readme file.

The backend and the client communicate with each other using HTTP REST calls synchronous requests.

As mentioned in the assignment description, the application is containerized by docker and deployed on a Kubernetes cluster.

Kubernetes Deployment :

There are two yaml files, *backend.yaml* and *client.yaml* that are created of type *ClusterIp*, that communicates with the backend and client by its DNS and not by its pod ip address, as you can see in the code snippet below.

```
apiVersion: v1
kind: Service
metadata:
```

```
name: backendservice
spec:
  type: ClusterIP
  selector:
    app: backendservice
  ports:
    - port: 8080
      targetPort: 8080
```

To create deployment and services:

```
kubectl apply -f k8s/backend.yaml
kubectl apply -f k8s/client.yaml
```

To verify:

```
kubectl get deployments
kubectl get pods
```

To restart:

```
kubectl rollout restart deployment backend-deployment
kubectl rollout restart deployment client-deployment
```

Baseline Control Group:

Client service: The client service is a simple springboot application that uses a minimal Rest Template to communicate with the backendservice via ***http://backendservice:8080/api/message***.

Backend service: The backend service simply returns a static message as acknowledgement.

Kubernetes Environment: Both the services are deployed once using replicas: 1

Tests performed:

The tests were performed via powershell and bash script windows. After building the project through maven and running the *mvn clean install -DskipTests*, the ports are forwarded using : `kubectl port-forward svc/backendservice 8081:8080` and `kubectl port-forward svc/client-service 8080:8080` to create a temporary tunnel from the local machine to service inside the kubernetes cluster . The command used to check the response is `curl http://localhost:8080/`

1. Steady state:

In the steady state, the response generated is:

```
$ curl http://localhost:8080/
{"backendResponse":{"message":"Hello from Backend (normal mode)"}, "fromClient": "ClientService"}
diviy@danielle_lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
```

In the code, to make the backend predictable and measurable, two control points were added: `/config/failure` and `/config/latency` were added in *MessageController.java*. A snippet of which is shown below:

```
@PostMapping("/config/failure")
public ResponseEntity<Map<String, Object>> setFailure(@RequestBody
Map<String, Object> config) {
    double rate = ((Number) config.getOrDefault("failure_rate",
0.0)).doubleValue();
    int status = ((Number) config.getOrDefault("status_code",
500)).intValue();

    failureRate.set(rate);
    failureStatusCode.set(status);

    return ResponseEntity.ok(Map.of(
        "message", "Failure configuration updated",
        "failure_rate", rate,
        "status_code", status
    ));
}
```

Keeping these configuration parameters two commands were used to test manual failure levels.

Failure configuration:

```
curl -X POST http://localhost:8081/config/failure \
-H "Content-Type: application/json" \
-d '{"failure_rate": 1.0, "status_code": 500}'
```

Healthy configuration:

```
curl -X POST http://localhost:8081/config/failure \
-H "Content-Type: application/json" \
-d '{"failure_rate": 0.0, "status_code": 500}'
```

2. 100% Failure:

After configuring the failure, when curl command is given it shows:

```
diviy@danielle_lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$ curl http://localhost:8080/
{"fromClient":"ClientService failed to get response from BackendService","error":"500 : \\\"{\\\"error\\\":\\\"Injected failure for testing\\\"}\\\""}
diviy@danielle_lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$
```

3. Latency Test:

```
curl -X POST http://localhost:8081/config/latency \
> -H "Content-Type: application/json" \
> -d '{"delay_rate": 1.0, "delay_ms": 8000}'
{"message":"Latency configuration updated","delay_rate":1.0,"delay_ms":8000}
```

```
diviy@danielle_lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$ curl http://localhost:8080/
{"fromClient":"ClientService failed to get response from BackendService",
"error":"I/O error on GET request for \\\"http://backendservice:8080/api/message\\\": Read timed out"}
```

Logs were taken account of using the command : *kubectl logs deployment/client-deployment*

```
diviy@danielle_lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$ kubectl logs deployment/client-deployment

  ____
 /  _ \
(  )_/
 \___/

.TomcatWebServer : Tomcat initialized with port 8080 (http)
.TomcatWebServer : Tomcat initialized with port 8080 (http)
2025-11-07T11:50:15.363Z INFO 1 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-11-07T11:50:15.363Z INFO 1 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.30]
2025-11-07T11:50:15.461Z INFO 1 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-11-07T11:50:15.472Z INFO 1 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 2362 ms
2025-11-07T11:50:16.430Z INFO 1 --- [main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 1 endpoint beneath base path '/actuator'
2025-11-07T11:50:16.558Z INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-11-07T11:50:16.570Z INFO 1 --- [main] c.e.c.client.ClientServiceApplication : Started ClientServiceApplication in 4.716 seconds (process running for 5.54
2)
```

OBSERVATIONS:

- Coupling trade-off: When the backend failed completely, the client blocked all calls, suggesting that the backend and client were tightly coupled and the client depends on the backend availability and latency. But the performance is quicker since there is less overhead.

Part B: Implementing Resilience Patterns:

1. Implemented Circuit Breaker :

The circuit breaker was implemented using Resilience4j. The circuit breaker aims to open the circuit when the backend fails repeatedly. Keeping in mind that subsequent calls would fail instantly without waiting for timeouts. After a period of time, the circuit breaker would move to a half-open state to test recovery. If the test succeeds, the circuit closes again & resumes normal operation.

2. Circuit Breaker Logs:

The circuit breaker logs are configured as follows: (Referred from Hints for solving Lab 3):

- WARN: BackendService failure. Failure count increasing: 3/5.
- ERROR: Circuit Breaker OPENING. Failing fast for next 30s.
- INFO: Circuit Breaker HALF-OPEN. Attempting one test call.
- INFO: Test call successful. Circuit Breaker CLOSING.

2.1 Healthy state:

```
$ curl http://localhost:8080/
{"fromClient":"ClientService failed to get response from BackendService","error":"I/O error on GET request for \"http://backendservice:8080/api/message\": Read timed out"}
diviy@danielle lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$ kubectl logs deployment/client-deployment

TomcatWebServer : Tomcat initialized with port 8080 (http)
2025-11-07T11:50:15.363Z INFO 1 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-11-07T11:50:15.363Z INFO 1 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.30]
2025-11-07T11:50:15.461Z INFO 1 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-11-07T11:50:15.472Z INFO 1 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 2362 ms
2025-11-07T11:50:16.430Z INFO 1 --- [main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 1 endpoint beneath base path '/actuator'
2025-11-07T11:50:16.558Z INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-11-07T11:50:16.570Z INFO 1 --- [main] c.e.client.ClientServiceApplication : Started ClientServiceApplication in 4.716 seconds (process running for 5.542)

diviy@danielle lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
```

2.2. Warning , Error and Info:

```
diviy@danielle lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$ kubectl logs deployment/client-deployment
2025-11-07T12:57:26.303Z INFO 1 --- [clientservice] [nio-8080-exec-1] c.example.client.service.BackendCli
Calling BackendService at http://backendservice:8080/api/message
2025-11-07T12:57:26.309Z WARN 1 --- [clientservice] [nio-8080-exec-1] c.example.client.service.BackendCli
CircuitBreaker fallback triggered: org.springframework.web.client.HttpServerErrorException$InternalServerError
500 : "{\"error\":\"Injected failure for testing\"}"
2025-11-07T12:57:26.431Z INFO 1 --- [clientservice] [nio-8080-exec-3] c.example.client.service.BackendCli
Calling BackendService at http://backendservice:8080/api/message
2025-11-07T12:57:26.437Z WARN 1 --- [clientservice] [nio-8080-exec-3] c.example.client.service.BackendCli
CircuitBreaker fallback triggered: org.springframework.web.client.HttpServerErrorException$InternalServerError
500 : "{\"error\":\"Injected failure for testing\"}"
2025-11-07T12:57:26.522Z WARN 1 --- [clientservice] [nio-8080-exec-5] c.example.client.service.BackendCli
CircuitBreaker fallback triggered: io.github.resilience4j.circuitbreaker.CallNotPermittedException: Circuit
er 'backendService' is OPEN and does not permit further calls
2025-11-07T12:57:26.658Z WARN 1 --- [clientservice] [nio-8080-exec-7] c.example.client.service.BackendCli
CircuitBreaker fallback triggered: io.github.resilience4j.circuitbreaker.CallNotPermittedException: Circuit
er 'backendService' is OPEN and does not permit further calls
2025-11-07T12:57:26.766Z WARN 1 --- [clientservice] [nio-8080-exec-9] c.example.client.service.BackendCli
CircuitBreaker fallback triggered: io.github.resilience4j.circuitbreaker.CallNotPermittedException: Circuit
er 'backendService' is OPEN and does not permit further calls
2025-11-07T12:57:26.865Z WARN 1 --- [clientservice] [nio-8080-exec-1] c.example.client.service.BackendCli
CircuitBreaker fallback triggered: io.github.resilience4j.circuitbreaker.CallNotPermittedException: Circuit
er 'backendService' is OPEN and does not permit further calls
2025-11-07T12:57:26.971Z WARN 1 --- [clientservice] [nio-8080-exec-3] c.example.client.service.BackendCli
CircuitBreaker fallback triggered: io.github.resilience4j.circuitbreaker.CallNotPermittedException: Circuit
er 'backendService' is OPEN and does not permit further calls
2025-11-07T12:57:27.075Z WARN 1 --- [clientservice] [nio-8080-exec-5] c.example.client.service.BackendCli
CircuitBreaker fallback triggered: io.github.resilience4j.circuitbreaker.CallNotPermittedException: Circuit
er 'backendService' is OPEN and does not permit further calls
2025-11-07T12:57:27.185Z WARN 1 --- [clientservice] [nio-8080-exec-7] c.example.client.service.BackendCli
CircuitBreaker fallback triggered: io.github.resilience4j.circuitbreaker.CallNotPermittedException: Circuit
er 'backendService' is OPEN and does not permit further calls
2025-11-07T12:57:27.280Z WARN 1 --- [clientservice] [nio-8080-exec-9] c.example.client.service.BackendCli
CircuitBreaker fallback triggered: io.github.resilience4j.circuitbreaker.CallNotPermittedException: Circuit
er 'backendService' is OPEN and does not permit further calls
```

3. Configurations with Retries:

3.1. Persistent Failure:

```
diviy@danielle_lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$ curl -X POST http://localhost:8081/config/failure \
  -H "Content-Type: application/json" \
  -d '{"failure_code": 500, "message": "Failure configuration updated", "status_code": 500}'
Run and Debug (Ctrl+Shift+D) 1.0, "status_code": 500}
diviy@danielle_lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$ for i in {1..10}; do
  echo "Request $i:"; curl -s http://localhost:8080/; echo; echo
done
Request 1:
{"error": "500 : \"\\\"error\\\": \"Injected failure for testing\\\"\", \"fromClient\": \"ClientService (retry exhausted or backend error)\""}

Request 2:
{"error": "CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 3:
{"error": "CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 4:
{"error": "CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 5:
{"error": "CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 6:
{"error": "CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 7:
{"error": "CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 8:
{"error": "CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 9:
{"error": "CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 10:
{"error": "CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

diviy@danielle_lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$
```

3.2. Half-open to closed:

```
2025-11-07T14:11:51.286Z INFO 1 --- [clientService] [ main] c.e.client.ClientServiceApplication : Starting ClientServiceApplication v1.0 SNAPSHOT using Java 17.0.16 wit
PID 1 (/app/app.jar started by root in /app)
2025-11-07T14:11:51.291Z INFO 1 --- [clientService] [ main] c.e.client.ClientServiceApplication : No active profile set, falling back to 1 default profile: "default"
2025-11-07T14:11:53.765Z INFO 1 --- [clientService] [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2025-11-07T14:11:53.788Z INFO 1 --- [clientService] [ main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-11-07T14:11:53.789Z INFO 1 --- [clientService] [ main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.30]
2025-11-07T14:11:53.847Z INFO 1 --- [clientService] [ main] o.a.c.c.C.[tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-11-07T14:11:53.850Z INFO 1 --- [clientService] [ main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 2481 ms
2025-11-07T14:11:55.934Z INFO 1 --- [clientService] [ main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 1 endpoint beneath base path '/actuator'
2025-11-07T14:11:56.021Z INFO 1 --- [clientService] [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-11-07T14:11:56.044Z INFO 1 --- [clientService] [ main] c.e.client.ClientServiceApplication : Started ClientServiceApplication in 5.333 seconds (process running for
5.852)
2025-11-07T14:13:34.324Z INFO 1 --- [clientService] [nio-8080-exec-1] o.a.c.c.C.[tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-11-07T14:13:34.324Z INFO 1 --- [clientService] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2025-11-07T14:13:34.262Z INFO 1 --- [clientService] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
2025-11-07T14:13:34.387Z INFO 1 --- [clientService] [nio-8080-exec-1] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:16:41.951Z INFO 1 --- [clientService] [nio-8080-exec-3] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:16:41.951Z WARN 1 --- [clientService] [nio-8080-exec-3] c.example.client.service.BackendClient : BackendService call failed after retries or due to error: org.springframework.web.client.HttpServerErrorException$InternalServerError: 500 : "{\"error\": \"Injected failure for testing\"}"
2025-11-07T14:18:56.292Z INFO 1 --- [clientService] [nio-8080-exec-5] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:19:30.539Z INFO 1 --- [clientService] [nio-8080-exec-7] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:19:30.554Z WARN 1 --- [clientService] [nio-8080-exec-7] c.example.client.service.BackendClient : BackendService call failed after retries or due to error: org.springframework.web.client.HttpServerErrorException$InternalServerError: 500 : "{\"error\": \"Injected failure for testing\"}"
2025-11-07T14:19:30.615Z ERROR 1 --- [clientService] [nio-8080-exec-9] c.example.client.service.BackendClient : CircuitBreaker is OPEN. Failing fast without calling BackendService.
2025-11-07T14:19:30.681Z ERROR 1 --- [clientService] [nio-8080-exec-1] c.example.client.service.BackendClient : CircuitBreaker is OPEN. Failing fast without calling BackendService.
2025-11-07T14:19:30.746Z ERROR 1 --- [clientService] [nio-8080-exec-3] c.example.client.service.BackendClient : CircuitBreaker is OPEN. Failing fast without calling BackendService.
2025-11-07T14:19:30.816Z ERROR 1 --- [clientService] [nio-8080-exec-5] c.example.client.service.BackendClient : CircuitBreaker is OPEN. Failing fast without calling BackendService.
2025-11-07T14:19:30.888Z ERROR 1 --- [clientService] [nio-8080-exec-7] c.example.client.service.BackendClient : CircuitBreaker is OPEN. Failing fast without calling BackendService.
2025-11-07T14:19:30.956Z ERROR 1 --- [clientService] [nio-8080-exec-9] c.example.client.service.BackendClient : CircuitBreaker is OPEN. Failing fast without calling BackendService.
2025-11-07T14:19:31.012Z ERROR 1 --- [clientService] [nio-8080-exec-1] c.example.client.service.BackendClient : CircuitBreaker is OPEN. Failing fast without calling BackendService.
2025-11-07T14:19:31.057Z ERROR 1 --- [clientService] [nio-8080-exec-3] c.example.client.service.BackendClient : CircuitBreaker is OPEN. Failing fast without calling BackendService.
2025-11-07T14:19:31.150Z ERROR 1 --- [clientService] [nio-8080-exec-5] c.example.client.service.BackendClient : CircuitBreaker is OPEN. Failing fast without calling BackendService.
```

3.3. Half-open to open:

```
diviy@danielle_lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$ for i in {1..6}; do
  echo "Trial $i:"; curl -s http://localhost:8080/; echo; echo
done
Trial 1:
{"fromClient": "ClientService here \u003D\u00C4B (circuit closed / retry OK)", "backendResponse": {"message": "Hello from Backend (normal mode)"}}

Trial 2:
{"fromClient": "ClientService here \u003D\u00C4B (circuit closed / retry OK)", "backendResponse": {"message": "Hello from Backend (normal mode)"}}

Trial 3:
{"fromClient": "ClientService here \u003D\u00C4B (circuit closed / retry OK)", "backendResponse": {"message": "Hello from Backend (normal mode)"}}

Trial 4:
{"fromClient": "ClientService here \u003D\u00C4B (circuit closed / retry OK)", "backendResponse": {"message": "Hello from Backend (normal mode)"}}

Trial 5:
{"fromClient": "ClientService here \u003D\u00C4B (circuit closed / retry OK)", "backendResponse": {"message": "Hello from Backend (normal mode)"}}

Trial 6:
{"fromClient": "ClientService here \u003D\u00C4B (circuit closed / retry OK)", "backendResponse": {"message": "Hello from Backend (normal mode)"}}
```

```

2025-11-07T14:18:56.292Z INFO 1 --- [client-service] [nio-8080-exec-5] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:19:30.539Z INFO 1 --- [client-service] [nio-8080-exec-7] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:19:30.554Z WARN 1 --- [client-service] [nio-8080-exec-7] c.example.client.service.BackendClient : BackendService call failed after retries or due to error: org.springframework.web.client.HttpServerErrorException$InternalServerError: 500 : "{\"error\":\"Injected failure for testing\"}"
2025-11-07T14:19:30.615Z ERROR 1 --- [client-service] [nio-8080-exec-9] c.example.client.service.BackendClient : CircuitBreaker is OPEN, Failing fast without calling BackendService.
2025-11-07T14:19:30.681Z ERROR 1 --- [client-service] [nio-8080-exec-1] c.example.client.service.BackendClient : CircuitBreaker is OPEN, Failing fast without calling BackendService.
2025-11-07T14:19:30.746Z ERROR 1 --- [client-service] [nio-8080-exec-3] c.example.client.service.BackendClient : CircuitBreaker is OPEN, Failing fast without calling BackendService.
2025-11-07T14:19:30.816Z ERROR 1 --- [client-service] [nio-8080-exec-5] c.example.client.service.BackendClient : CircuitBreaker is OPEN, Failing fast without calling BackendService.
2025-11-07T14:19:30.888Z ERROR 1 --- [client-service] [nio-8080-exec-7] c.example.client.service.BackendClient : CircuitBreaker is OPEN, Failing fast without calling BackendService.
2025-11-07T14:19:30.956Z ERROR 1 --- [client-service] [nio-8080-exec-9] c.example.client.service.BackendClient : CircuitBreaker is OPEN, Failing fast without calling BackendService.
2025-11-07T14:19:31.012Z ERROR 1 --- [client-service] [nio-8080-exec-1] c.example.client.service.BackendClient : CircuitBreaker is OPEN, Failing fast without calling BackendService.
2025-11-07T14:19:31.095Z ERROR 1 --- [client-service] [nio-8080-exec-3] c.example.client.service.BackendClient : CircuitBreaker is OPEN, Failing fast without calling BackendService.
2025-11-07T14:19:31.150Z ERROR 1 --- [client-service] [nio-8080-exec-5] c.example.client.service.BackendClient : CircuitBreaker is OPEN, Failing fast without calling BackendService.
2025-11-07T14:20:48.794Z INFO 1 --- [client-service] [nio-8080-exec-7] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:20:48.861Z INFO 1 --- [client-service] [nio-8080-exec-9] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:20:48.942Z INFO 1 --- [client-service] [nio-8080-exec-1] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:20:49.015Z INFO 1 --- [client-service] [nio-8080-exec-3] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:20:49.087Z INFO 1 --- [client-service] [nio-8080-exec-5] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message
2025-11-07T14:20:49.151Z INFO 1 --- [client-service] [nio-8080-exec-7] c.example.client.service.BackendClient : Calling BackendService at http://backendservice:8080/api/message

```

```

diviy@danielle lenovo MINGW64 ~/OneDrive/Desktop/ds-lab3/ds-lab3 (master)
$ for i in {1..10}; do echo "Request $i:"; curl -s http://localhost:8080/; echo; echo; done
Request 1:
{"error":"500 : \"\{\"error\": \"Injected failure for testing\"}\",\"fromClient\":\"ClientService (retry exhausted or backend error)\"}"

Request 2:
{"error":"500 : \"\{\"error\": \"Injected failure for testing\"}\",\"fromClient\":\"ClientService (retry exhausted or backend error)\"}"

Request 3:
{"error":"CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 4:
{"error":"CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 5:
{"error":"CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 6:
{"error":"CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 7:
{"error":"CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 8:
{"error":"CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 9:
{"error":"CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

Request 10:
{"error":"CircuitBreaker OPEN - backend temporarily disabled", "fromClient": "ClientService (circuit breaker FAST-FAIL)"}

```

NOTES ON EXPERIMENTS CONDUCTED ABOVE:

- Configuration of circuit breaker:
 - Max-attempts : 3
 - Wait-duration: 1s
 - Enable-exponential-backoff: true
- The client service continuously sent /GET/api/message per second, meanwhile the backendservice was alternately healthy and failing using the failure configuration and scaling to replica 0.

Process :

- The client began retrying failed requests , thereby persistent failures accumulated thus making the circuit breaker CLOSED -OPEN
- When the healthy config was restored, it moved from OPEN to Half-open and then finally to closed.

Observations:

- Retrying increased latency, once the circuit was open the responses were immediate.
- The circuit breaker automatically restored to closed once the backend stabilized.

After adding retries and jitters,

- The backend was configured to fail intermittently, handling 503 responses with retries.

●

OBSERVATIONS:

- Before jitters, the system was resilient and exhibited retry bursts under transient conditions.
- After adding the jitters, the retries became asynchronous, improving backend stability and reducing circuit breaker activations.

Retry trade-off: The implementation of retries, increased the system reliability during temporary backend issues, since it retried failed requests up to 3 times using exponential backoff, improving availability on the second or third attempt.

Latency trade-off: Without jitter, a retry storm would have been created., introducing it by trading off latency ensured temporary errors are recovered instead of visibly producing errors.

Stability vs Predictability trade off: Adding jitters improved system stability at the expense of predictable timing.

Part C- Chaos Engineering:

For the chaos engineering experiment, on one terminal the client log was started, the next terminal generated client load continuously and the third terminal experimented with killing and reviving the backend containers as attached along with the project repository.

